Storage and Retrieval of Aspects of Meaning in Directed Graph Structures

R. F. Simmons

System Development Corporation, Santa Monica, California

An experimental system that uses LISP to make a conceptual dictionary is described. The dictionary associates with each English word the syntactic information, definitional material, and references to the contexts in which it has been used to define other words. Such relations as class inclusion, possession, and active or passive actions are used as definitional material. The resulting structure serves as a powerful vehicle for research on the logic of question answering. Examples of methods of inputting information and answering simple English questions are given. An important conclusion is that, although LISP and other list processing languages are ideally suited for producing complex associative structures, they are inadequate vehicles for language processing on any large scale—at least until they can use auxiliary memory as a continuous extension of core memory.

1. Introduction

Behind the development of every new computer language there lies a set of problems and a set of programming structures with whose aid the problems can be managed. With FORTRAN the problem was to solve algebraic equations without the need for a great deal of I/O bookkeeping and without concern for detailed computer-word-packing statements. Behind JOVIAL lay the command-control problem, which customarily dealt with complex data structures and the need to use every bit of computer memory efficiently. IPL grew in response to a need to use associative list structures of nonnumeric symbolic data in a computer. LISP answered the need for a high-level functional language to handle recursive algebraic and symbolic structures. COMIT was the machine translator's approach to handling natural language strings.

In developing a special concept dictionary for storing and retrieving the meanings of words and phrases of English, the authors have found it desirable to use a complex network whose nodes are interrelated in several ways. Basically, the idea is that of a dictionary of English words in which each word has associated with it word-class information and lists of attribute-value pairs that define various aspects of its meaning and usage in terms of pointers to other words in the dictionary. In a data structure language such as JOVIAL, in addition to ordinary table structures several additional levels of associative coding are required to give easy access to the data without excessive costs in either space or processing time.

Because of the many levels of associative linking required the authors decided to use LISP, at least for early experimental work with the system. Advantages of LISP extended beyond the ease of producing complex data structures; they also included the simplicity of producing complex, often recursive functions for maintaining and querying the dictionary. An additional advantage is gained in the fact that although LISP is primarily an interpretive system it does allow for the compiling of a fast-running completed program. The most serious disadvantage of LISP for our system is that in present versions[1] it is limited to core memory for most uses. This limitation means that a dictionary of the type we are studying could not exceed two or three hundred words.

Since we are aiming for an eventual vocabulary of from five to fifty thousand words, the limitation to core memory is intolerable. Either an expansion of LISP will be required or the writing of a special language using auxiliary memory for handling cyclical structures in large complex networks will grow out of our experiments with the conceptual dictionary.

2. The Problem

The major shortcoming of all existing retrieval systems is their inability to handle anything vaguely resembling the meaning of words taken singly, let alone the meaning of the language strings that they comprise. Synonym dictionaries and thesauri have often been added but have proved but feeble makeshifts offering little improvement over the use of root forms of words alone. To the extent that automatic syntactic analysis has been available it has only emphasized the need for word and phrase meanings.

Five years of Synthex research toward the development of question-answering systems based on natural language text have confirmed this inability to deal with meanings. In these five years many approaches have been attempted toward representing some aspects related to the meaning of words. Most have been unsuccessful. It was learned early that the use of a synonym dictionary did not greatly improve our understanding of text in response to questions. More recently it was realized that even a well-coded thesaurus was not an answer. At vari-

Presented at an ACM Programming Languages and Pragmatics Conference, San Dimas, California, August 1965. Because of the author's illness, the paper was presented by John F. Burger, System Development Corporation.

This work is part of the Synthex Project supported by the Advanced Research Projects Agency, under Contract SD-97.

[1] In adapting LISP to the IBM 360 time-sharing design, means for accepting disk as a continuous extension of core are now being studied.

our times attempts were made to save syntactic contexts associated with words as possible representations of meanings; these too, although promising, did not appear to be a reasonable answer. With more recent research, particularly that of Bobrow [1964], Raphael [1964], Quillian [1965], and Thompson [1964], it has become apparent that, in addition to dictionary-type meanings, there is a need for something that can best be characterized as a knowledge of the world (e.g., Cows eat grass. Walls are vertical. Grass doesn't eat., etc.). Without something representing knowledge of the world it can hardly be hoped that a word or sentence can be understood.

The consequence of this line of thought is the realization that the problem requires the development of a conceptual dictionary that would contain definitional material, associative material, and some representation of knowledge of the world. These three aspects of a word's meaning seem to be the minimum that will allow for enough understanding of English to make question answering even a reasonable probability.

3. The Conceptual Dictionary

In a conceptual dictionary each word would be characterized by (a) a set of class memberships, (b) a set of attributes, (c) a set of associations, and (d) a set of active and passive actions.

The set of class memberships includes statements of the form the/an X(noun) is a/an Y(noun). Thus, "an aardvark is an animal" or "an aardvark is a mammal" are both examples of statements giving rise to class membership characteristics. For many nouns the class membership set is one of the basic definitional aspects of the word.

Attributes characterizing a word are such that if y is an attribute of x, then "x is y" is a true statement and the string "the yx" is grammatical. Thus if "scaly" is an attribute of "aardvark," then "an aardvark is scaly" is true and "the scaly aardvark" is grammatical. Associates of a word are in a loose part-whole relationship. If x has a y, then y is an associate of x; thus "John has a wallet" and "John has a nose" provide the two associates, "nose" and "wallet," for John.

The set of actions characterizing a word are derived from the verbs and their complements or objects that appear in context with the word. The two sentences "Natives eat aardvarks" and "Aardvarks eat ants" provide "eaten by natives" and "eat ants" as passive and active actions related to aardvarks.

The idea underlying this schema is that the meaning of a word can be conceptualized in bands of closeness of relation. The class membership of an object is so closely related as to be an integral part of it or its perception. Attributes and things closely associated with a word are seen as important but less essential, while the actions associating one word and another may range from important to irrelevant. Thus a man is an animal or he cannot be a man. "A man is ruddy" or "a man is handsome" are examples of attributes characterizing a particular man. But "a man has a nose" or "he has a wallet" or "he has high hopes" or "has an automobile" are examples that offer a wide range of associated facts. That the man lives, swims, plays golf, eats to live, etc., offers an accumulation of information about him that describes his behavior in the world. Figure 1 illustrates the author's view of these three relations.

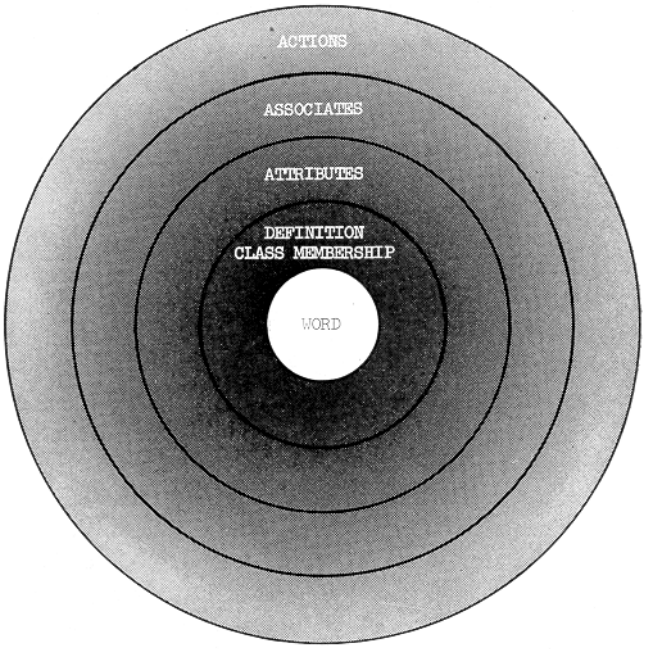

FIG. 1. Aspects of word and world meaning

It would be meaningful to consider additional aspects of word and world meaning beyond the four outlined, but those will serve at least as a beginning.

How to obtain and code the information about attributes, associations and actions is an important question. Quillian [1965], in a previous related system, codes by hand the information contained in a dictionary and reads it into a complex set of IPL list structures. Raphael [1964] in another related system inputs simple English sentences that are within a subset of English structures that his program can decode and translate into LISP functions. The authors, too, rely on English sentence inputs. Some of the sentences derive from dictionary definitions, some from ordinary text.

The present experimental LISP system[2] uses a grammar that parses any sentence that is an ordered subset of the following pattern:

Adj. string, Noun, Prep. Phrase, Verb, Adverb string, Prep. Phrase
Adj. string, Noun, Prep. Phrase.

This pattern obviously allows a considerable flexibility in simple English sentence construction. The parsing function will be revised to handle more complex patterns from time to time as these become desirable inputs. At the moment, problems of word-class ambiguity and pronominal reference have simply been ignored, but it is

[2] Programmed by John Burger.

```
ENTER(((HORSE . NOUN)(WAGON . NOUN)))
 DICTIONARY
PLURALS(((HORSE . HORSES)(WAGON . WAGONS)))
 OK
PULL(HORSE)
 (HORSE (POS . NOUN) (PL . HORSES))
LEARN((A HORSE IS AN ANIMAL))
 LEARNED
LEARN((HORSES ARE BIG AND STRONG))
 LEARNED
LEARN((HORSES SOMETIMES PULL BIG HEAVY WAGONS))
 LEARNED
PULL(HORSE)
 (HORSE (POS . NOUN)
   (PL . HORSES)
   (IS1 (((ANIMAL) (AN) NIL NIL)))
   (IS2 (BIG STRONG))
   (DO (((PULL) (SOMETIMES) NIL NIL ((WAGONS) (BIG HEAVY) NIL NIL)))))
PULL(WAGON)
 (WAGON (POS . NOUN) (PL . WAGONS) (HORIZ (HORSE)))
```

FIG. 2. Some of the input functions used in the system and their effect on the dictionary.

expected that with a dictionary of meanings available these problems can be dealt with successfully.

The parsing system transforms an English string such as "aardvarks have long prehensile tongues" into the format of the dictionary, essentially the following:

Attribute	*Value*
entry	aardvark
has	tongue (is2 long) (is2 prehensile)

The main list of the dictionary contains a Hollerith representation of each English word. Associated with the word is its plural form, its word-class and a list of attribute-value pairs of the type illustrated above. The various attributes include *is1*, the class-inclusion relation; *is2*, the adjectival modifying relation; *has*, the associative relation; *do*, the active or passive actions, and the various preposition names, *with*, *at*, *from*, etc., to indicate the various types of prepositional relationships. The values of these attribute-value pairs are the pointer to an English word that in itself may be further qualified by lists of pairs.

A second list associated with each dictionary entry is that of all its occurrences in the definitions of other words. This list allows easy access to uses of, for example, "aardvark" in the sentences in which it was not the subject. Thus, if the statement "African natives eat aardvarks" exists under the entry "natives," it can be recovered in the reference to "aardvarks" or, for that matter, to "eat."

The definitional list for each entry may be conceived of as a vertical structure that allows for an expansion of the idea represented by a word, while the list of occurrences is thought of as a horizontal access to additional usages or "knowledge of the world" associated with the word. In the present experimental system, only words that have been subjects of sentences will have vertical structure, but of course all words in the dictionary will have some horizontal structure. It is apparent that at least verbs will also need vertical structure and perhaps other parts of speech will as well, but, as yet, we have no way of producing such structure except by making explicit statements such as "to run is to move rapidly" or "hungry is a state of wanting food." (Note that the first of these two sentences is already beyond the present grammar of the system.) The essence of the approach is that every word or concept is characterized by a list of attribute-value pairs that include various important named relations of the word to other words or phrases, all of which must be in the dictionary, and a list of all occurrences of the word in other definitions.

Figure 2 shows the use of various functions to enter data for the dictionary and to learn statements. The function *PULL* (*X*) prints out the entire structure so far associated with *X*. This is illustrated for the words "horse" and "wagon." To represent a multiword concept in the dictionary is a relatively easy matter of assigning to it a number that is the name of a list in which the addresses of the series of words that make up the phrase are to be found. For example, the concept "African native" might be assigned the number 9250, which would be an ordinary entry in the dictionary but would signal the presence of a list in which the two words "African" and "native" are referred to. As a dictionary entry, "9250" has the usual syntactic and semantic information of a word.

Generating Statements From the Dictionary. To generate a statement from the concept dictionary (disregarding refinements of syntax) a few simple rules are required.

(1) A word may be conjoined to any of its attributes or classes by "is."

(2) A word may be conjoined to any of its associates by "has."

(3) A word may be conjoined to its 2-tuple actions; if these are passive the transform must be applied.

(4) Any attribute or associate may be further expanded by the action of "which is," "which has" or "which (action 2-tuple)" applied to sublists of that characteristic.

Additional rules of greater syntactic complexity are in fact added but these are sufficient to generate the following example sentences:

Aardvark is an animal.
Aardvark is African.
Aardvark eat ant.
Aardvark eat termite.
Native eat aardvark which is animal which eat ant.

Figure 3 is an example of a few sentences generated from the dictionary. It will be noticed that additional rules of agreement and for the assignment of articles have been included to improve the English structure of the generated output, but that further rules are required to make the English completely acceptable.

Answering Simple Questions. The general approach to answering simple questions of fact is one of using the content of the question to generate material related to the question word. For example, "What is an aardvark" can

```
OUTPUT1(AARDVARKS HAS)

 (AARDVARKS HAVE LONG STICKY TONGUES STRONG CLAWS AND BIG
  EARS WITH MUCH HAIR)

OUTPUT1(AARDVARKS DO)

 (AARDVARKS EAT ANTS EAT ALSO TERMITES AND DIG HOLES IN
  THE GROUND)

OUTPUT1(ACCORDION IS1)

 (ACCORDION IS A MUSICAL INSTRUMENT)
```

FIG. 3. Some sentences generated from the dictionary

```
QUERY((NAME 3 ANIMALS))

 (THE FOLLOWING ANIMALS ARE AMONG THOSE LEARNED- CAT
  DOG AARDVARK)

QUERY((WHAT DO COWS DO))

 (COWS EAT GRASS MAKE MILK AND CHEW THEIR CUD)

QUERY((DO AARDVARKS EAT ANTS))

 YES
```

FIG. 4. The system's answers to some simple questions

be used to generate the class memberships of "aardvark" as a proposed answer. Those attributes that are not nouns are rejected. In response to "What do aardvarks eat?" the system can respond with "ants" and "termites," rejecting the passive action of "being eaten by natives." "What is the difference between x and y?" can be answered by comparing characteristics of x and y. "What are several animals which eat grass?" can be answered by discovering several words which have an attribute of "animal" *and* 2-tuple action "eat grass." More difficult questions such as "Who was the second President of the United States?" require more complex processing but are still well within the capability of the system.

Figure 4 is an example of some simple questions and the system's answers to them. Questions, as every graduate student knows, can become remarkably complex in many different ways. In the consideration of ever more complex questions we can generally see how the system might be able to compute answers, but at ever-increasing costs of processing time and with increasingly complicated functions. In a practical sense, certain types of questions will remain for some considerable time beyond the capability of any system we are willing to build.

Paraphrasing Sentences. Within the limits of the grammar at any given time, the system offers the possibility of paraphrasing ordinary English statements. For example, if it is known that an aardvark is an animal that eats ants and termites, that an African native is a Negro from Africa, and that to eat is to ingest, the system can paraphrase "African natives eat aardvarks" to the prolix equivalent, "Negros from Africa ingest animals which eat ants and termites."

Mechanical Translation. The system strongly suggests that, by the development of two parallel dictionaries of the form described, the paraphrasing of one language may be done in the other language. However, much experimentation would be required to work out problems of ambiguity and correspondence between dictionaries.

4. Toward an Operational System

The conceptual dictionary briefly described above exists now as an experimental LISP program in 47k of core memory in the ARPA-SDC Q-32 Time-Sharing system. It is currently limited to a dictionary of 200–300 words and a relatively small set of program functions. What is required of even an early operational system is the ability to handle from 20 to 50 thousand words and a rather large set of functions for dealing with questions of increasing difficulty. Our expectations are that LISP will be expanded into a system that uses up to four million words of disk to augment core and that we will be able to pay an increased cost of response time in favor of continuing to use LISP for a large operational system. If that cost is prohibitive it will be necessary to produce a system tailored to the needs of the conceptual dictionary, and that will be able to use auxiliary memory efficiently to deal with a very large network of complexly linked words.

Although it is our belief that new problems create the need for new languages, it is apparent that existing languages are largely sufficient for our language processing problems, but in many cases, especially among the list-oriented languages, they simply have not geared themselves to the large amounts of data and data processing required in this special field.

Acknowledgments. I wish to acknowledge my debt to the twenty or so people who have studied question-answering systems over the past decade. Their work is reviewed elsewhere [Simmons 1965]. Readers knowing the Quillian system will recognize that the result of the author's three years of acquaintance with Quillian was the appropriation wholeheartedly of his ideas insofar as the author was able to understand them. A special debt is also expressed to Fred Thompson for leading the author to an understanding of parsing directly into a data structure and for acquainting him with TEMPO's forthcoming language system of associative cycles. Programming and detail design of the conceptual dictionary described in this paper were accomplished by John Burger.

REFERENCES

1. BOBROW, D. G. Natural language input for a computer problem-solving system. *Proc. Fall Joint Comput. Conf.* 1964, *25* (Also available as Ph.D. Thesis, Math. Dept., MIT, Cambridge, Mass., 1964.)
2. QUILLIAN, R. Word concepts: a theory and simulation of some basic semantic capabilities. Paper 79, CIT, Pittsburgh, Pa., April 5, 1965.
3. RAPHAEL, B. SIR: a computer program for semantic information retrieval. *Proc. Fall Joint Comput. Conf.* 1964, *25* (Also available as Ph.D. Thesis, Math. Dept., MIT, Cambridge, Mass., June 1964.)
4. SIMMONS, R. F. Answering English questions by computer: a survey. *Comm. ACM 8*, 1 (1965), 53–70.
5. THOMPSON, F. B., ET AL. DEACON breadboard summary. RM64TMP-9, TEMPO General Electric, Santa Barbara, Calif., March 1964.

Salton opened the discussion with the comment that a system such as this is inherently not extendable. The system will operate nicely with various kinds of "fish," but will run into trouble when "whales" appear, since the system will not know how to deal with aquatic mammals. He compared the system to the "Baseball" system, in which one cannot go beyond a limited range of questions, e.g., one has trouble if a new team appears. Young objected to this view, saying he believed the two systems were quite different, and that the present system was in principle infinitely extendable and very general. He said he could conceive of a system one level above this one which could deal with generalizations and which would make handling propositions easier than with the present special programming.

Burger said that work with higher level relationships was planned, but that the immediate extensions would be more trivial, in the way of inserting a great deal of knowledge about the world, of the kind any child has (e.g., "Walls are vertical.").

Gorn observed that the present system is based upon two forms of the verb "is" and the verb "has," and that more relationships were needed. Burger said the system was not in principle so limited. Gorn then asked how they would deal with the statement " 'Word' is a word." Burger had no immediate answer, though he thought they would eventually be able to deal with such cases.

Responding to a question from Cheydleur, Burger said they had about 50 different functions.

Mooers asked if there were any inherent features of LISP which limited their work. Burger said that a fundamental limitation was the limitation to use of core storage alone in the SDC version of LISP. They could not use disks. The second limitation was the inability to break out individual characters from the "atoms" of LISP. This prevents an easy and direct way of treating the similarity between "dog" and "dogs." At present this relationship has to be put in as a separate piece of information.

Mitchell then mentioned that for a very much larger data base a limitation will be the time required to pass the dictionaries through the machine. He said one of the big improvements in speed due to syntax-directed compilers was that they had less need to refer to a very large dictionary. Burger admitted that this was a very important problem, and one which is being considered. What can be done outside of LISP is being studied, since a problem in using an auxiliary memory with a list-structure system like LISP is that interrelationships between lists are broken up if just one section is brought from the auxiliary memory. So far, a good solution to the problem has not been found.

TRAC, A Procedure-Describing Language for the Reactive Typewriter

Calvin N. Mooers

Rockford Research Institute Inc., Cambridge, Massachussetts

A description of the TRAC (Text Reckoning And Compiling) language and processing algorithm is given. The TRAC language was developed as the basis of a software package for the reactive typewriter. In the TRAC language, one can write procedures for accepting, naming and storing any character string from the typewriter; for modifying any string in any way; for treating any string at any time as an executable procedure, or as a name, or as text; and for printing out any string. The TRAC language is based upon an extension and generalization to character strings of the programming concept of the "macro." Through the ability of TRAC to accept and store definitions of procedures, the capabilities of the language can be indefinitely extended. TRAC can handle iterative and recursive procedures, and can deal with character strings, integers and Boolean vector variables.

Presented at an ACM Programming Languages and Pragmatics Conference, San Dimas, California, August 1965; a supplementary paper, not on the announced program.

The present work was supported in part by the following grants and contracts: Advanced Research Projects Agency contract SD-295, Information Sciences Branch of the Air Force Office of Scientific Research contracts AF-AFOSR 376, 377 and 461-64, and Division of General Medicine, National Institutes of Health, U.S. Public Health Service grant GM 10416.

Introduction

The TRAC (Text Reckoning And Compiling) language system is a user language for control of the computer and storage parts of a reactive typewriter system. A reactive typewriter is understood to be one of a number of teletypewriters simultaneously connected online by wire to a memory and computer complex which permits real-time, multiple access (time-shared) operation. In the philosophy of the reactive typewriter, the man at the typewriter keyboard is the focal point of the system. The connected storage and computer devices are considered to be peripheral service units to the reactive typewriter.

The design goals for the TRAC language and its translating system included: (1) high capability in dealing with back-and-forth communication between a man at a keyboard and his work on the machine, so as to permit him to make insertions and interventions during the running of his work; (2) maximum versatility in the definition and performance of any well-defined procedure on text; (3) ability to define, store and subsequently use such procedures to extend the capabilities of the language; and finally (4) maximum clarity in the language itself so that it could be easily taught to others. A discussion of these design goals, and of the design decisions which went into